Data Analyst Agent

Ask questions about a CSV in plain English; get tables, charts and answers, with the code that produced them.

Project report

LLM: Groq · openai/gpt-oss-120b

Sample run: On sales.csv (1,500 rows)

Table of Contents

1. Business Understanding 3

- The problem 3
- Who it is for 3
- What it offers 3
- Example 3

2. Project Understanding 4

- How it works 4
- Tools and APIs 4
- The agent's steps 4
- The sandbox 5
- Calling the LLM 5

3. Architecture Diagram 6

- Components 6

4. Flowchart 7

5. Final Outcome 8

- The app 8
- Results at a glance 8
- Sample outputs 9
- Self-correction test 12

1. Business Understanding

The problem

Most teams keep their numbers in spreadsheets and CSV exports: sales, orders, stock, budgets. The questions people ask of them are simple ("which region sold most last quarter?", "how is revenue trending?"), but answering them takes pivot tables, formulas or Python. People without those skills wait for an analyst, and analysts spend much of their week on the same small, one-off requests. Dashboards help only with the questions someone planned for in advance.

Who it is for

- **Managers and business users** in sales, operations and finance who have data but not the time or skills to write formulas or code.
- **Small businesses** with no data team, who need quick answers from their own exports.
- **Data analysts** who want routine questions answered in seconds, with working pandas code to build on.
- **Students** learning pandas and Plotly, who can see the code behind every answer.

What it offers

- **Ask in plain English:** upload a CSV and type a question; no formulas or code needed.
- **The right format automatically:** a table, a single number, a sentence or an interactive chart.
- **Reliable answers:** if the generated code fails, the agent reads the error and fixes it, up to 3 times.
- **No guessed column names:** the agent sees the file's columns, types and sample rows on every question.
- **Nothing hidden:** the plan and the code behind each answer are one click away, so results can be checked.
- **Follow-up questions:** "now only 2024" builds on the previous answer, like talking to an analyst.

Example

A sales manager uploads the year's sales export and asks "monthly revenue trend". A line chart appears in a few seconds. They follow up with "now filter that to 2024 only" and get the 2024 chart, then ask "top 3 salespersons by quantity sold" and get a table, all without writing a formula.

2. Project Understanding

How it works

The app is written in Python. When a CSV is loaded, it is read into a pandas DataFrame and summarised into a short schema description (columns, types, sample rows, missing values, statistics). Each question then runs through an agent built with LangGraph. The agent sends the question and the schema to a large language model (LLM) through its API, gets back a plan and Python code, and runs that code in a restricted sandbox. If the code fails, the error goes back to the LLM to fix. The result is shown in a Streamlit web page.

Tools and APIs

Tool	Used for
Python	the whole project
Streamlit	the web interface: upload, chat, tables and charts
LangGraph	the agent's state machine: plan, generate, execute, fix
LangChain (langchain-groq, langchain-google-genai)	one interface for calling either LLM
Groq API	the default LLM, openai/gpt-oss-120b (about 2 seconds per call, free tier)
Google Gemini API	the alternative LLM (gemini-3.5-flash by default)
pandas, NumPy	loading the CSV and running the analysis
Plotly Express	interactive charts
python-dotenv	reading the API key from the .env file
pytest	? automated tests, run without an API key or network
fpdf2, Kaleido	this report, and the chart images in it

The agent's steps

Step	What it does
plan	Asks the LLM for a one or two sentence approach, given the schema, the question and recent history.
generate	Asks for Python that uses df, pd, px and np and stores the answer in RESULT. Markdown fences and stray prose are stripped from the reply.
execute	Runs the code in the sandbox on a copy of the data and classifies RESULT as a table, chart, value or text. A failure returns a trimmed traceback.
fix	Sends the traceback and the failed code back to the LLM for a corrected version; adds 1 to retries.
should_retry	Routes to fix while there is an error and fewer than 3 fixes have been tried; otherwise ends.

The sandbox

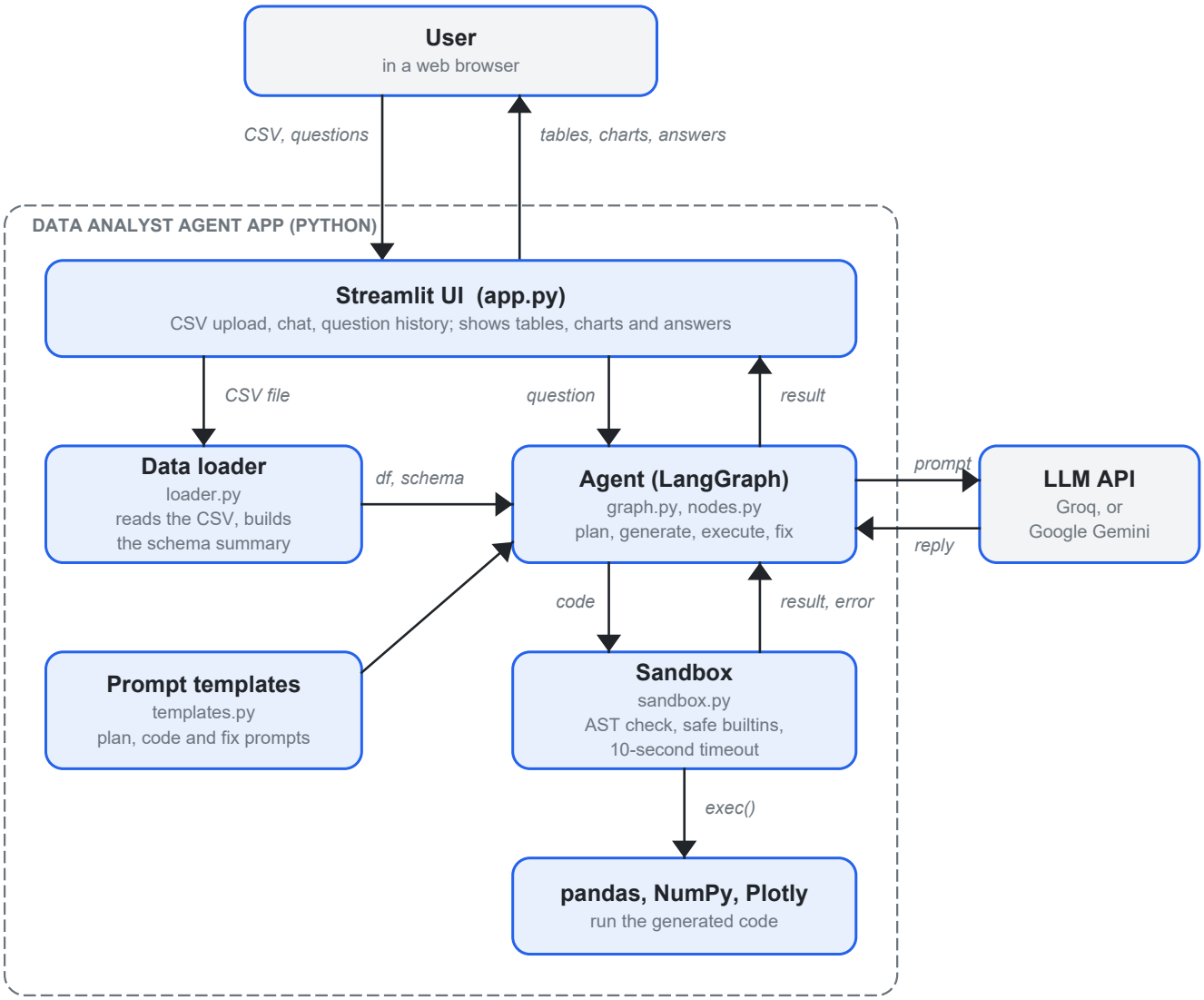
- An **AST check** rejects imports, file access, eval/exec and dunder attributes before anything runs, with a message the fix step can act on.
- Only **whitelisted builtins** are available (len, sum, sorted, round and similar), so open() and import do not exist.
- The code gets a **copy** of the DataFrame, so a bad in-place edit cannot change the user's data.
- A **10-second timeout** stops runaway loops from blocking the app.

Calling the LLM

One question costs at least two LLM calls (plan and generate), plus one per fix. The last five questions, with their code, are included so follow-ups work. Temporary failures (rate limits, server errors, timeouts) are retried up to three times, and the app tells the user it is waiting.

3. Architecture Diagram

The components of the system and what passes between them. Everything inside the dashed outline runs in one Python process; the LLM is an external API.

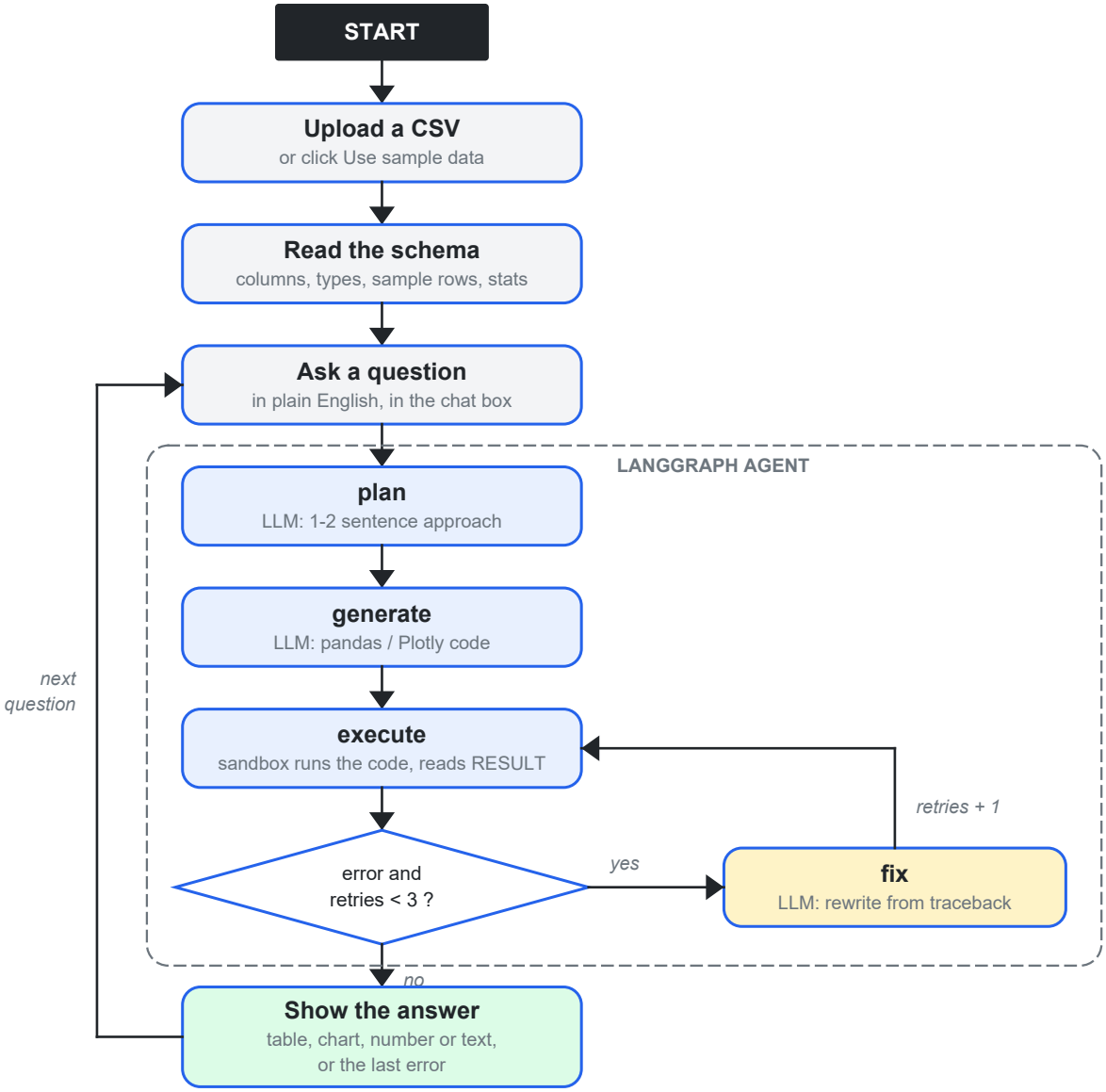


Components

Component	Responsibility
Streamlit UI (app.py)	File upload, chat input, question history; renders each result by its type.
Data loader (src/data/loader.py)	Reads the CSV (UTF-8 or latin-1), parses date columns, builds the schema summary sent with every prompt.
Agent (src/agent/)	The LangGraph state machine and its four nodes; ask() is the single entry point.
Prompt templates (src/prompts/)	The plan, code and fix prompts, plus the recent-history block.
Sandbox (src/execution/sandbox.py)	Checks and runs the generated code; returns the result or a traceback.
Settings (src/config.py, .env)	API keys, which LLM to use, retry and timeout limits.
LLM API (Groq or Gemini)	Writes the plan and the code; reached over HTTPS through LangChain.

4. Flowchart

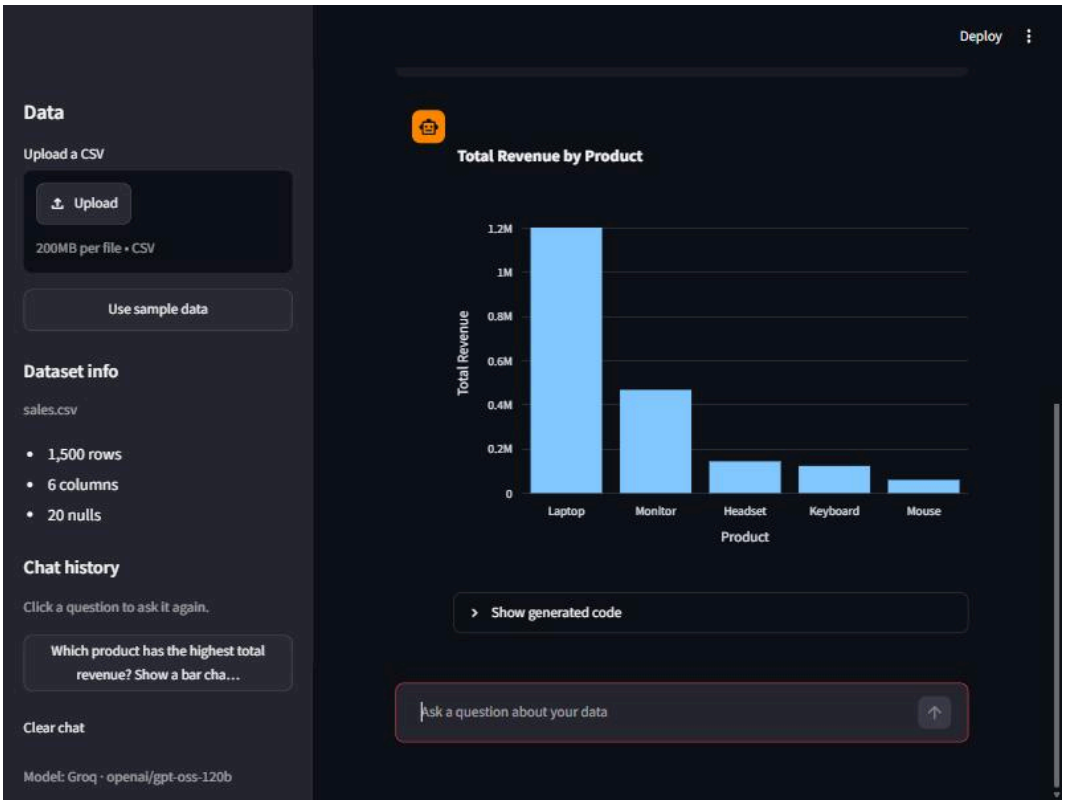
What happens from loading a file to seeing an answer. Grey steps happen in the app, blue steps are the agent, and the yellow step runs only when the code fails.



1. The user uploads a CSV, or picks the sample sales data.
2. The loader reads it and builds the schema summary.
3. The user types a question.
4. **plan:** the LLM describes the approach in one or two sentences.
5. **generate:** the LLM writes pandas or Plotly code that stores its answer in RESULT.
6. **execute:** the sandbox checks and runs the code.
7. If the code failed and fewer than 3 fixes have been tried, **fix** sends the error back to the LLM and the new code runs again.
8. The answer is shown as a table, chart, number or text, with the plan and code under it. The next question starts again at step 3, with this one as context.

5. Final Outcome

The app



Results at a glance

- **9 of 9** sample questions answered, **9** of them with the first code.
- The self-correction test recovered from a real KeyError in **1 fix attempt**.
- **?** automated tests pass, covering the loader, prompts, sandbox, agent loop and app.

#	Question	Answer	Fixes	Time
1	Show the first 5 rows	Table, 5 rows	0	2.2 s
2	What's the total revenue?	1,996,526.43	0	1.5 s
3	Revenue by region as a bar chart	Bar chart	0	1.8 s
4	Top 3 salespersons by quantity sold	Table, 3 rows	0	1.7 s
5	Monthly revenue trend	Line chart	0	2.2 s
6	Now filter that to 2024 only	Line chart	0	2.6 s
7	Which product has the highest average revenue?	Text answer	0	8.1 s
8	Compare regions with a pie chart	Pie chart	0	8.3 s
9	Show a heatmap of revenue by product and region	Heatmap	0	9.9 s
10	Total revenue by product, highest first	Table, 5 rows	1	5.7 s

Run on sales.csv (1,500 rows) with Groq (openai/gpt-oss-120b), 2026-10-31 08:39.

Sample outputs

Q1. "Show the first 5 rows"

0 fix attempts · 2.2 s

date	product	region	revenue	quantity	salesperson
2023-01-01	Laptop	West	4,624.51	5	Ben
2023-01-02	Mouse	South	60.80	2	Elena
2023-01-02	Laptop	South	4,619.01	5	Dev
2023-01-03	Mouse	East	180.02	6	Farid
2023-01-03	Mouse	East	30.29	1	Carla

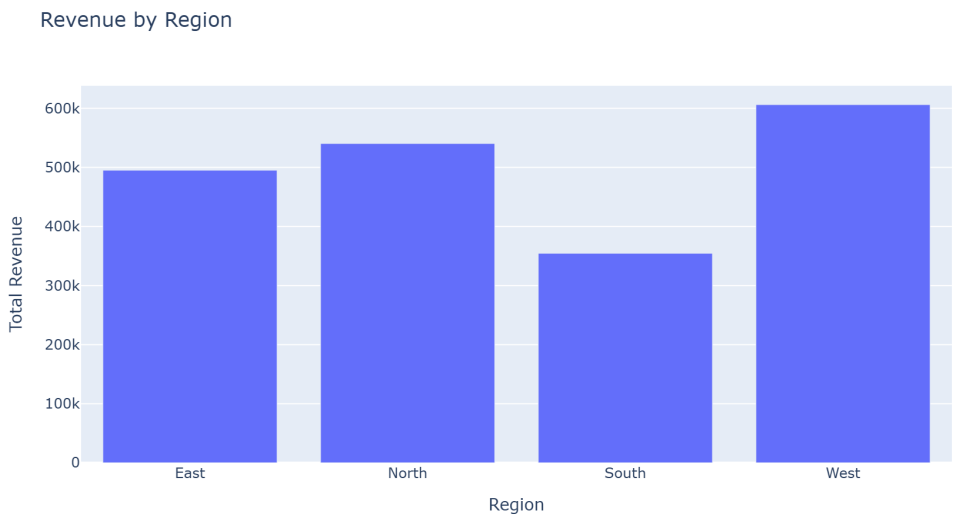
Q2. "What's the total revenue?"

0 fix attempts · 1.5 s

1,996,526.43

Q3. "Revenue by region as a bar chart"

0 fix attempts · 1.8 s



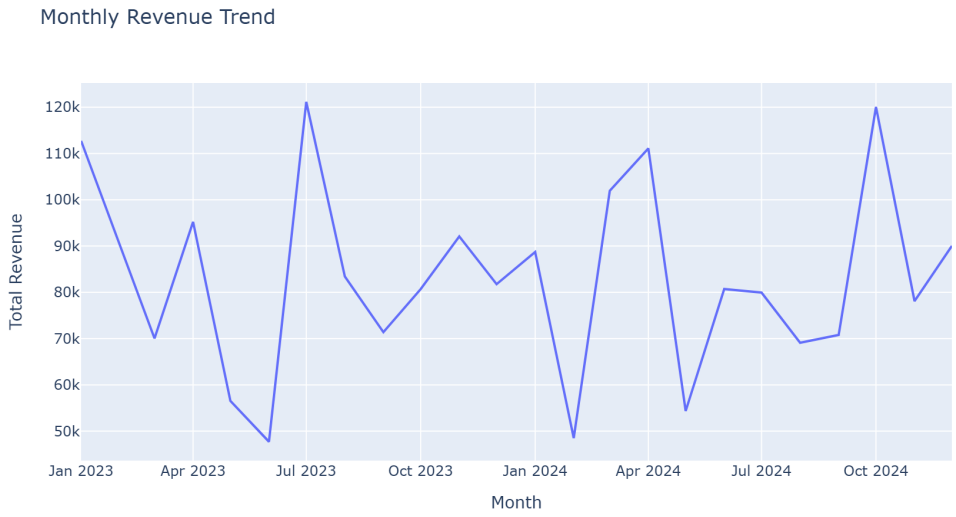
Q4. "Top 3 salespersons by quantity sold"

0 fix attempts · 1.7 s

salesperson	quantity
Ben	1,497
Asha	1,433
Farid	1,310

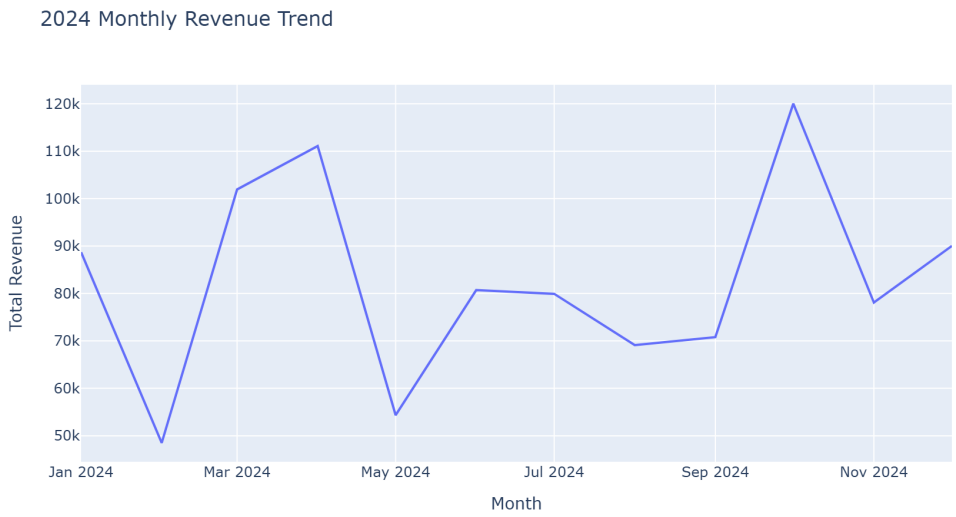
Q5. "Monthly revenue trend"

0 fix attempts · 2.2 s



Q6. "Now filter that to 2024 only"

0 fix attempts · 2.6 s · follow-up to the monthly trend



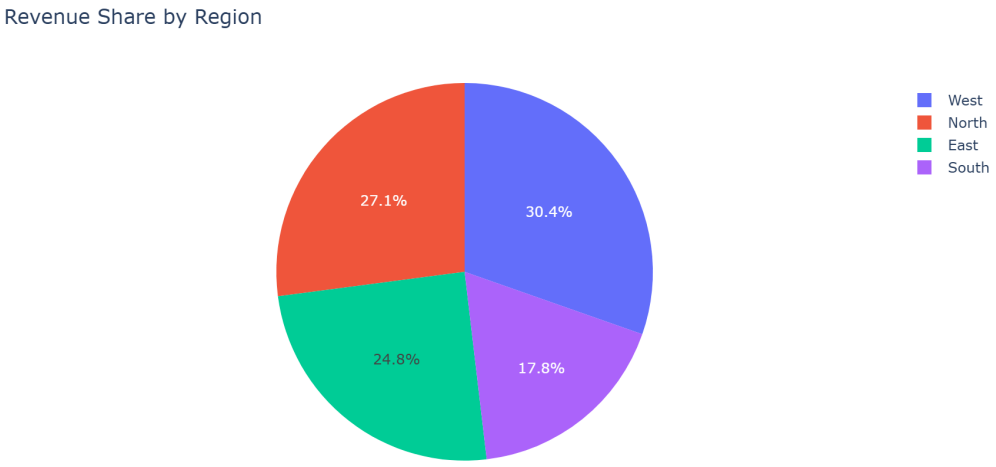
Q7. "Which product has the highest average revenue?"

0 fix attempts · 8.1 s

Laptop has the highest average revenue (5,031.41).

Q8. "Compare regions with a pie chart"

0 fix attempts · 8.3 s



Q9. "Show a heatmap of revenue by product and region"

0 fix attempts · 9.9 s



Self-correction test

For this test the first code was replaced with a version that misspells a column (revenu). The sandbox raised a real KeyError, and the live model's fix step read it and repaired the code.

FIRST CODE (BROKEN ON PURPOSE)

```
RESULT = df.groupby('product')['revenu'].sum().sort_values(ascending=False)
```

ERROR SENT BACK TO THE LLM

```
Traceback (most recent call last):
  File "<generated code>", line 1, in <module>
    RESULT = df.groupby('product')['revenu'].sum().sort_values(ascending=False)
             ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "pandas/core/base.py", line 241, in __getitem__
    raise KeyError(f"Column not found: {key}")
KeyError: 'Column not found: revenu'
```

CODE AFTER 1 FIX ATTEMPT

```
RESULT = df.groupby('product')['revenue'].sum().sort_values(ascending=False).reset_index()
```

RESULT

product	revenue
Laptop	1,202,506.99
Monitor	467,745.69
Headset	143,747.43
Keyboard	122,904.31
Mouse	59,622.01